



Instituto Politécnico Nacional

Escuela Superior de Computo



Materia: Aplicaciones para Comunicaciones en Red

Profesor: Tanibet Pérez De los Santos Mondragón

Integrantes: Gil Valentín Oswaldo

Grupo: 6CV1

Fecha: 21/03/2026

Actividad: Ejercicio 2.3 - Demultiplexación en TCP y análisis de operaciones bloqueantes

Introducción

El protocolo TCP permite la coexistencia de múltiples conexiones simultáneas hacia un mismo puerto de servicio. Este proceso, conocido como **demultiplexación**, se basa en la identificación única de cada conexión mediante la cuádrupla:

- Dirección IP origen
- Puerto origen
- Dirección IP destino
- Puerto destino

Desarrollo

1. Servidor base

Considere el siguiente servidor TCP:

```
#!/usr/bin python3
import socket
HOST = "localhost" #Direccion de la interfaz de loopback
estándar (localhost)
PORT = 5050 #Puerto que usa el cliente (los puertos sin
privilegios son > 1023)
buffer_size = 1024

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as TCPServerSocket:
    TCPServerSocket.bind(("0.0.0.0", PORT))
    TCPServerSocket.listen(5)
    print("El servidor TCP está disponible y en espera de solicitudes")
    while True:
        Client_conn, Client_addr = TCPServerSocket.accept()
        #Crear objeto thread (client_conn)
        with Client_conn:
            print("Conectado a", Client_addr)
            while True:
                print("Esperando a recibir
                datos... ") data =
                Client_conn.recv(buffer_si
                ze) print ("Recibido,",
                data," de ")
            if not data:
                break
            print("Enviando respuesta a", Client_addr)
            Client_conn.sendall(data)
```

2. Análisis de la demultiplexación

Al ejecutarse la instrucción: `conn, addr = server.accept()`

el sistema operativo:

- Acepta una nueva conexión entrante
- Realiza la demultiplexación correspondiente
- Crea un nuevo socket (`conn`) asociado exclusivamente a ese cliente

De esta forma, cada cliente conectado dispone de un canal de comunicación independiente, aun cuando todos se conecten al mismo puerto del servidor.

3. Ejecución experimental

1. Inicie el servidor.
2. Establezca al menos dos conexiones cliente simultáneas.
3. Envíe datos desde cada cliente.

4. Observación del comportamiento

Se observará que:

- El servidor atiende únicamente a un cliente a la vez
- Las conexiones adicionales no son procesadas hasta que la conexión activa finaliza

Este comportamiento no se debe a limitaciones del protocolo TCP, sino a la estructura del programa.

5. Análisis de operaciones bloqueantes

El flujo de ejecución del servidor está determinado por dos llamadas principales:

```
conn, addr = server.accept() data = conn.recv(1024)
```

Ambas son **operaciones bloqueantes**, lo que implica que:

- `accept()` suspende la ejecución hasta que se establece una nueva conexión
- `recv()` suspende la ejecución hasta que se reciben datos en la conexión activa

En consecuencia, el servidor:

- No puede aceptar nuevas conexiones mientras procesa una existente
- No puede atender otros clientes si el cliente actual no envía datos.

6. Modificación a modo no bloqueante

Modifique el servidor para utilizar sockets no bloqueantes:

```
server.setblocking(False)
```

y, tras aceptar una conexión:

```
conn.setblocking(False)
```

7. Nuevo comportamiento observado

Al ejecutar el servidor modificado, se observará que:

- a. Las llamadas a `accept()` y `recv()` generan excepciones cuando no hay conexiones o datos disponibles
- b. El programa deja de bloquearse, pero pierde la capacidad de gestionar adecuadamente el flujo de ejecución

Esto ocurre porque el modo no bloqueante requiere un mecanismo adicional para determinar cuándo un socket está listo para operar.

A partir de la práctica, se concluye que:

- c. TCP implementa correctamente la **demultiplexación de conexiones**
- d. Cada conexión es representada por un socket independiente en la aplicación
- e. Sin embargo, el uso de **operaciones bloqueantes** puede impedir la atención concurrente
- f. El uso de sockets no bloqueantes introduce la necesidad de mecanismos de sincronización adicionales

Conclusión

La demultiplexación es una característica propia del protocolo TCP y del sistema operativo. No obstante, el aprovechamiento efectivo de múltiples conexiones depende del diseño de la aplicación.

Para lograr concurrencia real en servidores TCP, es necesario incorporar mecanismos como:

- g. Multiprogramación (threads o procesos)
- h. Multiplexación de E/S (select, poll)
- i. Modelos asíncronos (asyncio)

Sección 1: Análisis de funciones bloqueantes

Actividad: Recrea el escenario ejecutando el servidor TCP iterativo y conectando múltiples clientes.

1. Describe el procedimiento que seguiste para recrear el escenario con múltiples clientes.

```
aplicaciones = [
    "Cliente 1: Spotify",
    "Cliente 2: Netflix",
    "Cliente 3: Youtube",
    "Cliente 4: Discord",
    "Cliente 5: WhatsApp"
]

def simular_cliente_tcp(nombre_app):
    # Creamos un socket TCP nuevo. Windows le asignará un puerto de origen único.
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as TCPClientSocket:
        try:
            TCPClientSocket.connect((HOST, PORT))

            # Extraemos el puerto local que Windows nos acaba de asignar para imprimirlo
            puerto_origen = TCPClientSocket.getsockname()[1]
            print(f"[{nombre_app}] Conectado desde el puerto local {puerto_origen}. Enviando mensaje...")

            # Armamos y enviamos el mensaje
            mensaje = f"Hola servidor, soy {nombre_app}"
            TCPClientSocket.sendall(mensaje.encode('utf-8'))

            # Esperamos la respuesta
            data = TCPClientSocket.recv(buffer_size)
            print(f"[{nombre_app}] Respuesta: {data.decode('utf-8')}")

        except ConnectionRefusedError:
            print(f"[{nombre_app}] Error: El servidor rechazó la conexión. ¿Está encendido?")
        except Exception as e:
            print(f"[{nombre_app}] Error inesperado: {e}")
```

Primero creé una lista de clientes para simular la conexión con 5 clientes distintos y se crea la función para la simulación de los 5 clientes.

```
# Lanzamos un hilo por cada aplicación
for app in aplicaciones:
    hilo = threading.Thread(target=simular_cliente_tcp, args=(app,))
    hilos.append(hilo)
    hilo.start()
    time.sleep(0.2)

# Esperamos a que todos los hilos terminen su trabajo
for hilo in hilos:
    hilo.join()
```

Después creamos un hilo para cada cliente y esperamos a que terminen de enviar el mensaje.

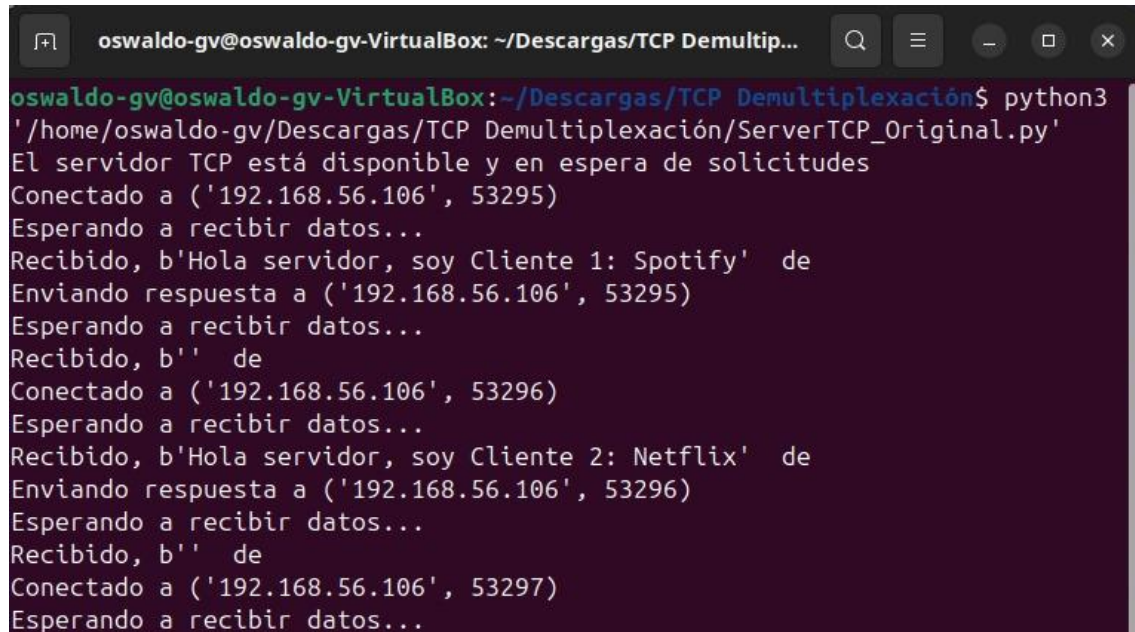
2. Explica el comportamiento observado cuando:

- **Un cliente está conectado y enviando datos**

El servidor acepta la conexión y lo atiende recibiendo el mensaje del cliente.

- **Un segundo cliente intenta comunicarse con el servidor**

El servidor atiende los clientes uno por uno en el orden en el que fueron llegando hasta terminar con los 5.



```
oswaldo-gv@oswaldo-gv-VirtualBox: ~/Descargas/TCP Demultip...
oswaldo-gv@oswaldo-gv-VirtualBox:~/Descargas/TCP Demultiplexación$ python3
'/home/oswaldo-gv/D.../TCP Demultiplexación/ServerTCP_Original.py'
El servidor TCP está disponible y en espera de solicitudes
Conectado a ('192.168.56.106', 53295)
Esperando a recibir datos...
Recibido, b'Hola servidor, soy Cliente 1: Spotify' de
Enviando respuesta a ('192.168.56.106', 53295)
Esperando a recibir datos...
Recibido, b'' de
Conectado a ('192.168.56.106', 53296)
Esperando a recibir datos...
Recibido, b'Hola servidor, soy Cliente 2: Netflix' de
Enviando respuesta a ('192.168.56.106', 53296)
Esperando a recibir datos...
Recibido, b'' de
Conectado a ('192.168.56.106', 53297)
Esperando a recibir datos...
```

En esta imagen podemos ver como va atendiendo a cada cliente conforme va llegando.

3. Identifica las funciones bloqueantes en el código del servidor.

```
Client_conn, Client_addr = TCPServerSocket.accept()
```

La primera es la función de accept().

```
data = Client_conn.recv(buffer_size)
```

La segunda función es recv().

4. Para cada función identificada, explica:

- **En qué condiciones se bloquea**

Para accept(): El servidor espera solicitudes de conexión, pero si no hay ninguna solicitud de conexión no puede seguir ejecutándose el resto del código.

Para recv(): Si el cliente se conecta el servidor ahora espera a que envíe datos, pero si el cliente no manda nada el servidor continúa esperando.

- **Cómo afecta la atención de otros clientes**

Dado que el servidor atiende cliente por cliente esto se volvería muy tardado al querer conectarse múltiples clientes a la vez.

5. Describe un escenario específico en el que el servidor queda completamente bloqueado.

Como el servidor en estas condiciones solo atiende cliente por cliente, si hay algún cliente que cumpla alguna de las dos condiciones anteriores el servidor se quedará esperando hasta que se elimine el proceso.

Sección 2: Sockets no bloqueantes

Actividad: Modifica el servidor para trabajar en modo no bloqueante.

1. Indica qué modificaciones realizaste al código.

```
HOST = "192.168.56.102"
PORT = 5050
buffer_size = 1024
clientes_activos = []
```

Creamos una lista de clientes que están conectados.

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as TCPServerSocket:
    TCPServerSocket.bind((HOST, PORT))
    TCPServerSocket.listen(5)
    #Modo no bloqueante
    TCPServerSocket.setblocking(False)
```

Configuramos el servidor en modo no bloqueante.

```
print(f"Servidor en cámara lenta iniciado en el puerto {PORT}. Observando excepciones...\n")
while True:
    # CAZANDO EXCEPCIÓN A: accept()
    try:
        Client_conn, Client_addr = TCPServerSocket.accept()
        Client_conn.setblocking(False)
        print(f"\n¡NUEVA CONEXIÓN ACEPTADA DE {Client_addr}!")
        clientes_activos.append((Client_conn, Client_addr))

    except BlockingIOError as e:
        # Esto se imprimirá constantemente mientras nadie se conecte
        print(f"[Fase accept] Nadie intentando conectarse: Excepción invisible del sistema: {repr(e)}")
```

Añadimos en la parte de la conexión con el cliente para que esté en modo no bloqueante y cuando se acepte una conexión nueva se registre en la lista de clientes activo, y para capturar la excepción añadimos un print para saber la excepción que esta ocurriendo si es que se llega a presentar.

```

# CAZANDO EXCEPCIONES B y C: recv()
for conn, addr in clientes_activos[:]:
    try:
        data = conn.recv(buffer_size)
        if data:
            print(f"=== Dato recibido de {addr}: {data.decode('utf-8').strip()} ===")
            conn.sendall(data)
        else:
            print(f"El cliente {addr} cerró la conexión formalmente.")
            conn.close()
            clientes_activos.remove((conn, addr))

    except BlockingIOError as e:
        # Esto se imprimirá cuando el cliente esté conectado pero en silencio
        print(f"[Fase recv] {addr} conectado pero sin enviar datos: Excepción invisible: {repr(e)}")

    except ConnectionResetError as e:
        # Esto se imprimirá si cierras el cliente a la fuerza (Ej. cerrando su terminal de golpe)
        print(f"\n[Fase recv] ¡ALERTA! {addr} destruyó la conexión abruptamente: Excepción capturada: {repr(e)}")
        conn.close()
        clientes_activos.remove((conn, addr))

print("-" * 50)
time.sleep(5)

```

Creamos el bucle donde el servidor recibe la información de los clientes.

Primero espera a recibir los datos enviados por el cliente, cuando termina se cierra la conexión con ese cliente y se elimina de la lista de clientes activos.

Después, en la parte de las excepciones, se activan cuando se cumpla alguna de las condiciones, como que el cliente este conectado, pero no envía nada o cuando se termina la conexión de golpe. Para identificarlas imprimimos un mensaje con el nombre de la excepción ocurrida

Al final añadimos un sleep de 5 segundos para ver el comportamiento de forma mas pausada.

2. Ejecuta nuevamente el servidor y describe el comportamiento observado.

```

oswaldo-gv@oswaldo-gv-VirtualBox:~/Descargas/TCP Demultiplexación$ python3 '/home/oswaldo-gv/D
e/oswaldo-gv/Descargas/TCP Demultiplexación/ServerTCP_SNB.py'
Servidor en cámara lenta iniciado en el puerto 5050. Observando excepciones...

[Fase accept] Nadie intentando conectarse: Excepción invisible del sistema: BlockingIOError(11, 'Resource temporarily unavailable')
-----
[Fase accept] Nadie intentando conectarse: Excepción invisible del sistema: BlockingIOError(11, 'Resource temporarily unavailable')
-----

```

Con ayuda de la captura de la excepción 1 vemos que el servidor en efecto, está en un proceso de escucha activa en bucle. Esta es la Excepción A.

```

¡NUEVA CONEXIÓN ACEPTADA DE ('192.168.56.106', 61812)!
=== Dato recibido de ('192.168.56.106', 61812): Hola servidor, soy Cliente 1: Spotify ===

[Fase recv] ¡ALERTA! ('192.168.56.106', 61812) destruyó la conexión abruptamente
: Excepción capturada: ConnectionResetError(104, 'Connection reset by peer')
-----

¡NUEVA CONEXIÓN ACEPTADA DE ('192.168.56.106', 61813)!
=== Dato recibido de ('192.168.56.106', 61813): Hola servidor, soy Cliente 2: Netflix ===

[Fase recv] ¡ALERTA! ('192.168.56.106', 61813) destruyó la conexión abruptamente
: Excepción capturada: ConnectionResetError(104, 'Connection reset by peer')
-----

```

La segunda excepción que se puede generar es cuando hay un cliente conectado, pero sin enviar ningún mensaje. Esto lo logra simplemente comentando la línea del código del cliente original donde se envía un mensaje al servidor. Esta es la Excepción B

```

TCPClientSocket.connect((HOST, PORT))
print("Enviando mensaje...")
#TCPClientSocket.sendall(b"Hola servidor TCP ")
print("Esperando una respuesta...")
data = TCPClientSocket.recv(buffer_size)

```

```

¡NUEVA CONEXIÓN ACEPTADA DE ('192.168.56.106', 61812)!
=== Dato recibido de ('192.168.56.106', 61812): Hola servidor, soy Cliente 1: Spotify ===

[Fase recv] ¡ALERTA! ('192.168.56.106', 61812) destruyó la conexión abruptamente
: Excepción capturada: ConnectionResetError(104, 'Connection reset by peer')
-----

¡NUEVA CONEXIÓN ACEPTADA DE ('192.168.56.106', 61813)!
=== Dato recibido de ('192.168.56.106', 61813): Hola servidor, soy Cliente 2: Netflix ===

[Fase recv] ¡ALERTA! ('192.168.56.106', 61813) destruyó la conexión abruptamente
: Excepción capturada: ConnectionResetError(104, 'Connection reset by peer')
-----

```

La tercera excepción ocurre cuando el cliente se desconecta de forma abrupta, esto lo logramos simplemente cerrando la terminal donde ejecutamos el cliente modificado donde hacemos las 5 conexiones.

3. Identifica las excepciones que se generan durante la ejecución:

Excepción A: Al aceptar conexiones

Nombre de la excepción: BlockingIOError

En qué instrucción ocurre: `Client_conn, Client_addr = TCPServerSocket.accept()`

En qué condiciones se produce: Ocurre cuando el bucle principal da una vuelta y el servidor pregunta "¿Hay alguien nuevo queriendo conectarse?", pero en ese momento no hay ninguna solicitud de conexión pendiente en la fila del sistema operativo.

Excepción B: Al recibir datos

Nombre de la excepción: BlockingIOError

En qué instrucción ocurre: `data = conn.recv(buffer_size)`

En qué condiciones se produce: Ocurre cuando el servidor revisa a un cliente que ya está conectado y le pregunta "¿Me enviaste algo nuevo?", pero el cliente todavía no ha mandado ningún paquete de datos en ese instante.

Excepción C: La desconexión abrupta

Nombre de la excepción: ConnectionResetError.

En qué instrucción ocurre: `data = conn.recv(buffer_size)` o al intentar hacer `sendall()`.

```
data = conn.recv(buffer_size)
if data:
    print(f"=== Dato recibido de {addr}: {data.decode('utf-8').strip()} ===")
    conn.sendall(data)
```

En qué condiciones se produce: Cuando el cliente cierra su programa a la fuerza (ej. cerrando la terminal de golpe) sin hacer el proceso formal de despedida de TCP (enviar paquetes FIN). El servidor intenta leer o escribir, y el sistema operativo le avisa que la conexión fue destruida.

4. Explica por qué estas excepciones ocurren en modo no bloqueante.

En el modo normal (bloqueante): Si el servidor llama a `accept()` o `recv()` y no hay nadie conectándose o no hay datos, el sistema operativo pausa (bloquea) el programa. Y no lo desbloquea hasta que llega algo.

En el modo no bloqueante: En este modo se le dice al sistema operativo que no lo pause bajo ninguna circunstancia. Si le pide datos y no tiene nada en este momento exacto, se lo dice de inmediato para que el programa pueda seguir con otras cosas".

5. Describe qué problema nuevo aparece al eliminar el bloqueo.

Al eliminar el bloqueo, el ciclo while True: del servidor ya no se detiene a descansar en ningún momento. El programa entra en un bucle muy rapido haciendo las mismas preguntas a la máxima velocidad que permite el procesador:

¿Hay clientes nuevos? No.

¿El cliente 1 me envió algo? No.

¿El cliente 2 me envió algo? No.

(Y se repite muchas veces muy rapido).

Como consecuencia de esto es un alto consumo ineficiente de recursos.

Sección 3: Propuesta de mejora con concurrencia

Actividad: Analiza alternativas para permitir la atención simultánea de múltiples clientes.

1. Explica por qué el servidor actual no logra concurrencia real, aunque TCP soporte múltiples conexiones.

El servidor original (con sockets bloqueantes) no logra concurrencia porque tiene una arquitectura iterativa y secuencial, impulsada por un bucle anidado que lo "atrapa". Debido a que las funciones son bloqueantes, el programa se queda congelado ahí adentro platicando con el Cliente 1. El servidor físicamente no puede regresar a la línea de accept() para dejar pasar al Cliente 2 hasta que el Cliente 1 se desconecte por completo.

2. Propón una solución basada en concurrencia real (por ejemplo: threads, procesos o modelo asíncrono).

Para lograr una concurrencia real utilizando el modelo de Hilos, necesitamos dividir los procesos del servidor. En lugar de que un solo proceso intente hacer todo.

El Hilo Principal: Su única función será estar en un bucle infinito ejecutando TCPServerSocket.accept(). Su trabajo es solo recibir clientes nuevo

Los Hilos Secundarios: Cada vez que el Hilo Principal acepta a un cliente nuevo, en lugar de atenderlo él mismo, crea a un nuevo Hilo Secundario y le pasa el socket de ese cliente.

Ese nuevo hilo se encargará del bucle while True para hacer los recv() y sendall() con ese cliente en específico. Mientras tanto, el Hilo Principal ya regresó instantáneamente a la puerta (accept()) para recibir al siguiente.

3. Describe:

- **Cómo cambiaría el flujo del servidor**

El flujo de trabajo pasaría de ser secuencial a ser en paralelo

- **Qué ventajas aporta tu propuesta**

Dado de a que el flujo de trabajo ahora seria en paralelo los procesos se podrían realizar todos realmente al mismo tiempo ya que cada hilo se encarga de atender a cada cliente y no uno por uno.

- **Qué posibles desventajas introduce**

Tal vez podría ser el mayor consumo de recursos porque se están ejecutando múltiples procesos al mismo tiempo.

4. Incluye un fragmento de pseudocódigo o código que ilustre tu propuesta.

```
# 1. Definimos la función que será el Hilo Secundario
def manejar_cliente(conn, addr):
    print(f"[NUEVO HILO] Atendiendo la conexión de {addr}")
    with conn:
        while True:
            try:
                # Este hilo SÍ se bloquea aquí esperando a su cliente específico
                data = conn.recv(buffer_size)

                if not data:
                    print(f"[{addr}] El cliente cerró la conexión formalmente.")
                    break

                print(f"[{addr}] Recibió: {data.decode('utf-8').strip()}")
                conn.sendall(data)

            except ConnectionResetError:
                print(f"[{addr}] ¡Conexión perdida abruptamente!")
                break
```

```
def main():
    # 2. El Hilo Principal configura el servidor
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as TCPServerSocket:
        TCPServerSocket.bind((HOST, PORT))
        TCPServerSocket.listen(5)

    print(f"Servidor TCP Multihilo en espera de solicitudes en el puerto {PORT}...")

    while True:
        Client_conn, Client_addr = TCPServerSocket.accept()

        # Se crea un hilo nuevo para atender al cliente
        # Le pasamos a la función 'manejar_cliente' el socket y la IP que acabamos de aceptar
        hilo_cliente = threading.Thread(target=manejar_cliente, args=(Client_conn, Client_addr))

        # arranca el hilo en paralelo y el Hilo Principal regresa inmediatamente al accept()
        hilo_cliente.start()
```